07 — Konversasyonel Katman: Yazışarak İş Yapma + Onay Döngüsü

> **Bağlam (v1.1):** Bu doküman **Faz 8 — Konversasyonel Katman**'ın felsefesi ve UX yaklaşımını anlatır. Risk skoru + onay matrisi detayı: `09-otomasyon-esikleri.md`. Güncel resmî yapı: `proje-teklifi/02-cozum-genel-bakis.md`. Faz 8 ön koşulu: Faz 2-7 olgunlaşması.

Hipotez

> Hem müşteri hem yönetici, ERP ile **yazışarak** iş yapabilir. Sistem her aksiyondan önce **anladığını geri özetler**, kullanıcı onaylar veya revize eder, sonra aksiyon gerçekleşir, ardından sonuç geri rapor edilir. Bu çift yönlü döngü, klasik form-tabanlı arayüze ek (yedek değil) bir kanal olur.

OpenClaw'den alınan ilham (kopya değil)

OpenClaw VPS'i bize şu pattern'leri gösterdi:

Pattern	Bizdeki karşılığı
Persona katmanı (kimlik + üslup + sınır)	Paspas'ta her arayüz için ayrı persona: müşteri portalında "Paspas Sipariş Asistanı", yönetici panelinde "Operasyon Asistanı"
Çoklu provider routing (Anthropic/OpenAI/Groq)	Hızlı sohbet → Groq, derinlikli analiz → Claude, JSON yapılandırma → OpenAI
Komut → onay → eylem zinciri	"Sipariş yarat" komutu önce **taslak** oluşturur, kullanıcı onaylayınca DB'ye yazılır
Bildirim kanalı (Telegram)	Aynı pattern, ama opsiyonel; admin'in tercihine göre web push, email, Telegram
SSH/Repo otomasyon	Bizdeki karşılık: "ERP action" — siparişe dönüştür, üretime aktar, satın alma aç
Geri besleme döngüsü	Her aksiyon sonrası "tamamlandı / şu sapma var / X kayıt etkilendi" özeti

Önemli ayrım: OpenClaw **dışarıdan komuta veren** bir katman. Paspas'taki konversasyonel katman **içeride** çalışan, üstelik **müşteriye de açılan** bir katman. Yani daha içsel, daha yetki-bilinçli.

Üç ana persona

1. Müşteri Asistanı (B2B portal — public)

Kim: mevcut/yeni müşteri. Web/mobil B2B portala login olmuş.

Konuşabildikleri: - "Geçen ay aldığım Star Siyah'tan 200 takım daha istiyorum, ay sonuna yetişsin" - "Sipariş #SS-2026-0123 ne durumda?" - "Bu üründen 500 takım kaç güne kadar gelir, fiyatı nedir?" - "Sevkiyat geldi, eksik var mı kontrol edebilir miyim?"

Asistan ne yapar: - Sipariş taslağı çıkarır → "Anladığım: 200 takım STAR SİYAH (kod 1101 101), termin 30 Nisan, son fiyatınız 510₺/takım. Onaylıyor musunuz?" - Müşteri "evet" → sipariş gerçekten oluşur - Müşteri "termini 5 Mayıs yap" → taslağı revize eder, tekrar özet gösterir - Sipariş durumu → mevcut DB'den gerçek zamanlı çeker - **Asla** asistan tek başına sipariş açamaz, fiyat değiştiremez, kredi limiti aşamaz

Sınırlar: - Sadece o müşterinin verisine erişir (kendi siparişleri, kendi fiyat listesi) - Yetki dışı talep → "Bunu için satış ekibi sizi arasın mı?" yönlendirmesi - Şüpheli durumda (büyük tutar, alışılmadık miktar) otomatik insan onayına yönlendirir

2. Yönetici Asistanı (admin panel — internal)

Kim: Paspas yöneticisi/operasyon ekibi.

Konuşabildikleri: - “Bu hafta hangi makineler boş?” - “Star Siyah’ın son 30 günde stok düşüş hızı?” - “Kamil Tekstil’den son 6 ay neler aldı?” - “Şu 5 siparişi makinelere optimal nasıl dağıtırım?” - “Hammadde ‘X’ tedarikçi alternatif çıkar”

Asistan ne yapar: - Veriyi çeker, görselleştirir - Aksiyon önerirse onay bekler: “3 sipariş için üretim emri açayım mı?” → onaylanırsa açar - Karmaşık planlama → çoklu adım: “Önce stok kontrol edeyim → tedarikçiye sor → fiyat dönerse satın alma taslağı aç” - Her adım sonunda durum raporu

3. Saha Asistanı (operatör tablet/mobil — opsiyonel faz 2)

Kim: üretimde operatör, mal kabul personeli.

Konuşabildikleri: - “Bu makinede şu an ne üretiyoruz?” - “Kaç tane yaptım bugün?” - “Bu makinedeki kalıp arıza, bildirim gönder” - “Vardiya sonu, son 250 adet girişimi yap”

Mevcut operatör arayüzünün üzerine yazışma katmanı. Faz 1’de zorunlu değil.

Onay döngüsü — anatomik

Her etkileşim 5 aşamalı:

```
[1] KULLANICI → ham mesaj
       │
       ▼
[2] AI INTENT EXTRACTION
       │  Niyetin ne, hangi domain (sipariş/stok/müşteri vs.),
       │  hangi parametreler eksik?
       ▼
[3] AI ACTION PROPOSAL (taslak)
       │  "Yapacağım şey: ..."
       │  Yapılandırılmış JSON + insan dilinde özet
       ▼
[4] KULLANICI → onay / revize / iptal
       │  Onay → eyleme geç
       │  Revize → 2'ye dön (yeni parametrelerle)
       │  İptal → kapat
       ▼
[5] EYLEM GERÇEKLEŞTİRME + GERİ RAPOR
       │  ERP API'sini çağır, sonucu özetle
       │  "Sipariş SS-2026-0124 oluşturuldu, termin 30 Nisan"
       │  Gerekirse hata: "stok yetersiz, alternatif: X"
```

Kritik kural: [3] olmadan [5] asla olmaz. AI doğrudan eyleme geçemez. Düşük-risk eylemler (örn. “raporu göster”) onay aşamasını atlayabilir; yan etkili her şey (DB’ye yazma, mail gönderme, sipariş açma) onay zorunlu.

Mimari katmanlar

Katman 1: Konuşma altyapısı

```
CREATE TABLE konusma_oturumlari (
  id char(36) PK,
  persona ENUM('musteri','yonetici','operator'),
  kullanici_id char(36),                 -- müşteri.id veya users.id
  baslik varchar(255),                    -- AI özeti
  durum ENUM('aktif','tamamlandi','iptal'),
  baslangic datetime,
  son_etkilesim datetime,
  toplam_token int,
  toplam_maliyet decimal(10,4)
);

CREATE TABLE konusma_mesajlari (
  id char(36) PK,
  oturum_id char(36) FK,
  rol ENUM('user','assistant','system','tool'),
  icerik TEXT,
  intent JSON,                            -- {"domain": "siparis", "action": "olustur", ...}
  proposed_action JSON,                   -- onay bekleyen aksiyon
  approved tinyint,                       -- 1=onaylandı, 0=reddedildi, NULL=henüz
  executed_action_id char(36),            -- gerçekleşen aksiyon kaydı
  tokens_input int,
  tokens_output int,
  provider varchar(32),
  model varchar(128),
  latency_ms int,
  created_at datetime
);

CREATE TABLE konusma_eylemleri (
  id char(36) PK,
  mesaj_id char(36) FK,
  eylem_tipi varchar(64),                -- 'siparis.olustur', 'satin_alma.taslak', 'rapor.goster'
  payload JSON,
  sonuc JSON,
  durum ENUM('basarili','kismen','hata'),
  hata_mesaji varchar(2000),
  created_at datetime
);
```

Katman 2: Intent + Action engine

`konusma_engine` : - Mesajı + son N mesajı + persona context'i + erişilebilir tool listesini AI'a gönderir - AI iki olası çıktı: (a) sadece bilgi yanıtı, (b) yapılandırılmış action proposal - Action proposal `proposed_action` field'ına yazılır, kullanıcıya gösterilir

Tool registry (whitelist edilmiş ERP eylemleri):

```
type ERPTool = {
  id: string;                      // "siparis.olustur"
  description: string;             // AI prompt'a verilecek
  parameters_schema: ZodSchema;    // strict validation
  required_role: 'musteri'|'yonetici'|'operator';
  side_effects: 'none'|'read'|'write'|'irreversible';
  needs_approval: boolean;         // side_effects != 'none' ise true zorunlu
  handler: (params, context) => Promise<Result>;
};
```

Örnek tool'lar: - `siparis.olustur` (write, approval) - `siparis.durum_sorgu` (read, no approval) - `urun.fiyat_sorgu` (read, no approval) - `uretim_emri.olustur` (write, approval, sadece yönetici) - `satin_alma.taslak` (write, approval, sadece yönetici) - `stok.rapor` (read, no approval) - `lead.kayit` (write, approval, sadece yönetici) - `mail.gonder` (irreversible, approval, sadece yönetici)

Katman 3: Persona yönetimi

OpenClaw pattern'i: her persona ayrı `system_prompt` + ton + sınır seti.

```
CREATE TABLE konusma_personalari (
  id char(36) PK,
  kod varchar(64) UNIQUE,        -- 'musteri_asistani', 'yonetici_asistani'
  ad varchar(128),
  system_prompt text,
  default_provider varchar(32),
  default_model varchar(128),
  izinli_tool_ids JSON,           -- ["siparis.olustur","siparis.durum_sorgu"]
  ton_kurallari text,             -- "Resmi, kısa, müşteriye 'siz' kullanma"
  sinirlar text,                  -- "Fiyat değiştirme yetkisi yok"
  is_active tinyint
);
```

Mevcut `test_center_ai_templates`'dan farklı: bu **dialog-spesifik**, action engine'e bağlı. Yeniden kullanım tartışılabilir; önce ayrı tablo, sonra birleştirme düşünülür.

Katman 4: Bildirim + dış kanal

OpenClaw pattern'inden — telegram zorunlu değil: - Web push (admin paneli açıkken anlık) - Email (özet günlük rapor) - Telegram (isteyen yönetici için) - Whatsapp Business (ileride müşteri tarafı için)

Bu kanallar **sadece bildirim** — gelen mesaja içeriden cevap verilmez (faz 2'de düşünülür). Konversasyon Paspas'ın kendi UI'ı içinde.

Geri besleme — her etkileşim öğrenir

```
CREATE TABLE konusma_geri_besleme (
  id char(36) PK,
  mesaj_id char(36) FK,
  kullanici_skoru tinyint,        -- 1-5
  manual_duzeltme text,           -- AI yanlış anladıysa kullanıcının düzeltmesi
  intent_dogru tinyint,           -- AI'ın anlama doğruluğu
  action_uygunluk tinyint,        -- önerinin uygunluğu
  notlar text,
  created_at datetime
);
```

Haftada bir cron: - Düşük skorlu yanıtları çıkar - Persona'nın `system_prompt`'una "şu durumlarda şöyle yap" eklentisi öner - LLM'den few-shot örnek üret - Yönetici onayıyla persona güncellenir

OpenClaw ile bağlantı (orkestrasyon)

OpenClaw VPS'i orchestrator olarak kalır: - Cron: "günlük 06:00 yönetici özeti üret + email gönder" - Webhook: müşteri portala mesaj geldiğinde Paspas backend → OpenClaw'a forward → Paspas DB'ye yazıştırma kaydı - AI provider routing: OpenClaw'in çoklu provider config'ini Paspas backend'inden tekrar yazmamak için ortak proxy

Ama **konuşma DB'si Paspas'ta** kalır (domain veri orada). OpenClaw stateless yardımcıdır.

Use case akışı — bir baştan sona örnek

Müşteri B2B portala girer, sohbet açar:

> "Selam, geçen ay aldığım STAR SİYAH'tan 200 daha lazım. Mart sonuna yetişir mi?"

Sistem (intent extract):

```
{
  "domain": "siparis",
  "action": "olustur_taslak",
  "urun_ipucu": "STAR SİYAH",
  "miktar": 200,
  "termin": "2026-03-31",
  "musteri_id": "<oturumdan>",
  "guven": 0.85
}
```

Sistem (proposal): > “Anladığım: **STAR SİYAH (kod 1101 101)**, **200 takım**, termin **31 Mart**. Son fiyatınız **510₺/takım**, toplam **102.000₺ + KDV**. Mevcut üretim sırasına göre tahmini hazırlık süresi **9 iş günü**, yani 26 Mart’ta sevke hazır olur. Onaylar mısınız?”

Müşteri: “Evet ama 250 yapsak?”

Sistem (revize): > “Tamam, **250 takım** olarak güncellendim. Yeni toplam **127.500₺ + KDV**, hazırlık süresi **11 iş günü** (28 Mart’ta hazır). Onaylar mısınız?”

Müşteri: “Onayla”

Sistem (action): - `siparis.olustur` tool çağrılır - Validation: müşterinin kredi limiti, ürün stok takibi, üretim kapasitesi - DB’ye yazılır → SS-2026-0125 - Geri rapor: “Sipariş **SS-2026-0125** oluşturuldu. Termin tarihiniz **31 Mart**. Mail ile teyit gönderildi. Üretim başladığında size haber vereceğim.”

Sistem (otomatik takip): - Üretim başladığında → push bildirim - Sevkiyat hazırlandığında → push bildirim - Sevk edildiğinde → push bildirim - Hata/gecikme → erken uyarı

Açık sorular

1. **Müşteri portalı:** Şu an yok. Faz 1’de yönetici asistanı (admin panel iç chat) yapılır, portal sonra mı?
2. **Action onay UI:** Modal mı, inline kart mı? Mobil için?
3. **Konuşma geçmişi saklama:** Müşteri bazlı sınırsız mı, 6 ay sonra arşiv mi? KVKK
4. **Çoklu turn maliyeti:** Her mesajda son 10 mesaj + tool listesi gönderiliyor → token maliyeti büyür. Özet/sıkıştırma stratejisi gerek mi?
5. **Yetki modeli:** Müşteri sadece kendi siparişlerini görebilir — bu DB seviyesinde RLS (Row-Level Security) gerektirir mi, yoksa application kontrol yeterli mi?
6. **Halüsinasyon:** AI olmayan ürün/müşteri uydurursa? → Tool çağrısı validation katmanında DB’de kontrol; uydurma → kullanıcıya “böyle bir kayıt bulunamadı” mesajı.

7. **Çoklu dil:** Müşteri yurt dışıysa İngilizce/Almanca/Arapça konuşma desteği? Persona tek mi (otomatik dil algılama), persona başına dil mi?
8. **Sesli mesaj:** Whatsapp'tan ses geldiğinde STT (speech-to-text) ile metne çevirip işleme alma? Faz 3.

Bağımlılıklar

- ✅ `test_center/ai_provider.ts` — multi-provider altyapısı hazır
- ✅ Paspas DB'sinde `musteriler`, `urunler`, `siparis_*` mevcut
- ⚠️ Tool registry yok — yeni iş kalemi
- ⚠️ B2B portal frontend yok (müşteri tarafı için)
- ⚠️ Konuşma DB tabloları yok — yeni schema
- ⚠️ Onay UI/UX kuralları henüz tasarlanmadı

Faz planı

Faz	İçerik	Süre
A — yönetici asistanı v1	Admin panel içi chat sidebar, 5-10 read-only tool (rapor, sorgu)	~7-10 gün
B — yönetici asistanı v2	Write tool'lar (sipariş aç, üretim emri aç) + onay UI	~5-7 gün
C — geri besleme + retro	Skor + cron + persona update önerisi	~3-4 gün
D — müşteri portalı v1	Public B2B portal + read-only sohbet (sipariş durum, fiyat sorgu)	~10-14 gün
E — müşteri portalı v2	Sipariş açma + onay döngüsü	~5-7 gün
F — saha asistanı	Operatör mobil chat	~7-10 gün

Toplam (A-E): ~30-40 iş günü.

Vizyondaki yeri

Bu katman **5. motor** olarak `00-vizyon.md`'ye eklenmeli. Diğer 4 motora yatay bağlanır: - **Talep motoru:** sohbetten gelen yapılandırılmış talepler aynı havuza - **Müşteri/tedarikçi keşfi:** "şu lead'i çıkar" komutu - **Stok/üretim tahmin:** "bu hafta stok riski olan ürünleri özetle" - **Geri besleme:** her sohbetin sapma raporu — modeli güçlendirir

Konversasyonel katman **görünür arayüz**; arkasında 4 motor çalışır. Müşteri ne motor olduğunu bilmez, sadece "asistanla konuştuğunu" hisseder.